



Guardrail Fundamentals

LLM 應用的安全防護層:風險、架構與框架選型

AI/ML Engineer 學習系列 · 第一課

沒有防護的 LLM, 會出什麼事?

LLM 的輸出本質上是機率性的——原型可以容忍, 生產環境不行



幻覺 Hallucination

一本正經地捏造事實、亂編引用來源,
直接輸出給使用者就是事故



個資外洩 PII Leak

回應中夾帶姓名、電話、Email 等敏感資料,
觸犯隱私法規與內部政策



Injection 與 Jailbreak

惡意輸入劫持模型行為,
讓它無視系統指令、執行攻擊者的意圖



離題與濫用

客服機器人被拿來寫作業、聊政治,
燒你的 token、傷你的品牌

Guardrail 是什麼?

一個站在 LLM 與外界之間的「驗證層」:偵測風險、攔截或修正,再放行



檢查沒過的時候,Guard 可以怎麼做?



攔下 Exception

直接中止,回傳錯誤或預設回覆——高風險場景的預設



修正 Fix

自動遮罩 PII、裁掉違規片段,修完再放行



重問 Reask

把失敗原因回饋給 LLM,請它重新生成一次

Guard 的三個掛載點

不同位置的檢查,防的是不同階段的災難



Input Guard

在昂貴的 LLM 呼叫「之前」快速驗證使用者輸入

- 擋 injection、離題、惡意內容
- 省下被浪費的 token 與延遲
- 通常用便宜、快速的檢查



Output Guard

在回覆送出「之前」檢查 LLM 的最終輸出

- 擋幻覺、PII、違規內容
- 驗證結構化輸出符合 schema
- 是使用者體驗的最後防線



Tool Guard

包住有「副作用」的工具呼叫,執行前後都驗證

- 寄信、下單、改資料庫前先攔
- Multi-agent 時把驗證放在工具旁
- 別只依賴第一層 input guard

常見的檢查類型

各框架的內建檢查大同小異——這六種是共同的基本盤



Moderation 內容審查

偵測仇恨、暴力、色情等有害內容,通常接現成分類器



Injection / Jailbreak 偵測

辨識試圖劫持模型的輸入,例如「忽略以上指令」



離題偵測

確保對話待在允許的主題範圍內,不被拿去做別的事



PII 偵測與遮罩

找出姓名、電話、Email 等個資,攔下或自動遮罩



幻覺偵測

對照你的知識庫或來源文件,驗證回覆有憑有據



結構化輸出驗證

用 schema / regex 保證輸出格式正確,能被下游程式使用

三大主流框架:定位其實錯開了

不是「哪個最好」,而是「各自解決什麼問題」

Guardrails AI

驗證器派

用可組合的 Validator 攔截輸入輸出,並強制結構化輸出

- Guard + Validator + Hub 生態
- Provider-agnostic,不綁供應商
- 支援 Python 與 JavaScript
- Pydantic 風格,上手直覺

適合:結構化輸出、通用驗證層

NeMo Guardrails

對話流程控制派

用 Colang DSL 撰寫 rails,宣告主題邊界與對話流程

- 對話控制最靈活、最細緻
- 可接 LlamaGuard 等外部偵測器
- 學習曲線較陡,需要維護 rails
- NVIDIA 生態,開源免費

適合:Chatbot 的主題與流程管控

OpenAI Guardrails

開箱即用派

直接替換 OpenAI client,每次呼叫自動跑檢查

- Drop-in + no-code Wizard 配置
- Tripwire 機制:觸發即中止
- 與 Agents SDK 原生整合
- 綁定 OpenAI 生態是主要限制

適合:OpenAI 系應用快速上防護

深入一層:Guardrails AI 的核心物件

四個概念就能讀懂它的所有文件



Guard

守門員本體:掛在 LLM 呼叫外面,串起所有檢查



Validator

單一檢查邏輯:一個 validator 只驗一種風險



Hub

預建 validator 的市集:PII、毒性、regex...裝了就用



OnFailAction

失敗策略:exception / fix / reask,依風險等級選

```
from guardrails import Guard, OnFailAction
from guardrails.hub import RegexMatch
```

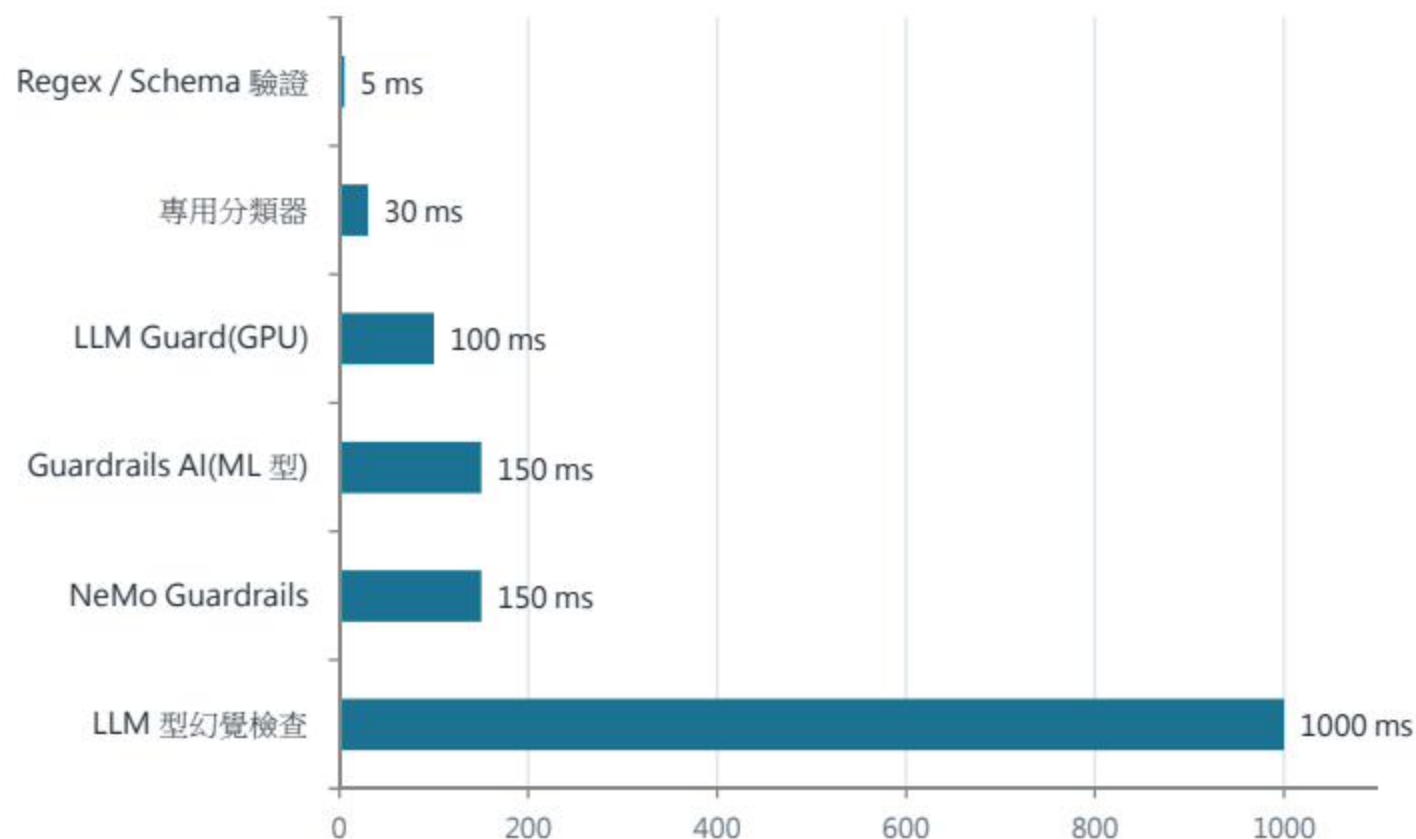
```
guard = Guard().use(
    RegexMatch,
    regex=r"\d{3}-\d{3}-\d{4}",
    on_fail=OnFailAction.EXCEPTION,
)
```

```
guard.validate("415-555-0198")
# -> pass
guard.validate("hello world")
# -> ValidationError
```

代價:每一層檢查都是延遲

安全不是免費的——用毫秒和 token 換來的(數字為量級示意,依硬體與配置而異)

各類檢查的典型延遲(毫秒)



50–100ms 是甜蜜點

多數團隊認為這個總開銷在生產環境可接受



平行執行壓延遲

互相獨立的檢查同時跑,別串成一條鏈



LLM 型檢查最貴

留給高風險路徑;每次檢查都是一次模型呼叫

縱深防禦:生產環境的真實架構

沒有任何單一層是 100% 有效——所以業界的答案是「疊起來」

第一層|輸入篩查

快速、便宜的 scanner 先過濾已知攻擊模式與明顯違規



第二層|對話 / 流程控制

主題邊界與流程規則,讓 injection 就算穿過也無法驅動行為



第三層|輸出驗證

最後防線:幻覺、PII、結構化輸出,確保送出去的東西乾淨



實務上的常態

- 多數生產 agent 組合 2-3 個工具,不是單選題
- 常見堆疊:快速 scanner + NeMo 管對話 + Guardrails AI 驗輸出
- 分層的目的:每一層攔不同類型的失敗

你的學習路線

按可遷移性排序:先學概念最通用的,再學生態綁定的



Capstone:縱深防禦 Demo

FastAPI 服務疊兩層防護(輸入篩查 + 輸出驗證)· React dashboard 即時顯示每層攔截了什麼· PostgreSQL 記錄攔截事件——三個學習目標在同一個專案匯流

帶走三件事



Guardrail 是 LLM 與世界之間的驗證層

掛在輸入、輸出、工具呼叫三個位置,失敗時可以攔、修、或重問



每一層檢查都有價目表

從幾毫秒的 regex 到秒級的 LLM 檢查——安全是用延遲與 token 買的



框架不是單選題,縱深防禦才是常態

輸入篩查 + 對話控制 + 輸出驗證,各攔各的失敗類型

下一課:安裝 Guardrails AI,動手寫你的第一個 Validator